

A Scalable Layer–Propagation and MRV Branching Method for Solving and Generating $N \times N$ Sudoku

Shaimaa Said Soltan

Abstract

This paper presents a general Sudoku solving and puzzle-generation framework capable of handling arbitrary grid sizes $N \times N$, including large orders such as 12×12 , 16×16 , 25×25 , and 36×36 . The method is based on a hybrid algorithm combining layer-based constraint propagation, single-missing deduction, minimum-remaining-values (MRV) heuristics, shallow branching, and early conflict detection. The solver is fully consistent with the implementation provided in the attached code, and the generator uses symmetry-preserving and structure-aware clue removal to ensure solvability even for large grids. We analyze the novelty of this approach and compare its performance to classical backtracking and human-style logical solvers.

1 Introduction

Sudoku solving algorithms traditionally fall into two categories:

- **Logical solvers**, which apply human-style deduction rules such as naked singles, hidden singles, pairs, triples, and advanced pattern recognition.
- **Backtracking solvers**, which perform depth-first search over candidate assignments until a valid solution is found.

Logical solvers do not scale well to large N , because the number of patterns grows combinatorially. Pure backtracking becomes infeasible for $N > 9$ due to the exponential search space. The solver described in this paper, based directly on the attached code, combines propagation and limited branching in a way that preserves both speed and generality.

2 Grid Structure and Box Shapes

For any valid Sudoku size N , the grid is partitioned into subregions of size $a \times b$, where $ab = N$. The implementation automatically selects the box shape (a, b) that is closest to a square by minimizing

$$|a - \sqrt{N}|.$$

Examples include:

$$9 = 3 \cdot 3, \quad 12 = 3 \cdot 4, \quad 16 = 4 \cdot 4, \quad 36 = 6 \cdot 6.$$

This generality allows the solver to support any N for which a rectangular factorization exists.

3 Layer-Based Propagation

The core innovation of the solver is the *layer construction* for each digit. For a given digit d , a boolean layer matrix is built indicating which empty cells *could* contain d after blocking:

- all cells in any box already containing d ,
- all cells in any row containing d ,
- all cells in any column containing d .

If a box has exactly one remaining allowed position for d , the digit is placed. This captures many classical logical rules in a single unified mechanism.

4 Single-Missing Deduction

After layer propagation, the solver performs deterministic filling of:

- any row with exactly one missing value,
- any column with exactly one missing value.

This is a powerful deduction step that often resolves large portions of the grid.

5 MRV Candidate Selection

When propagation stalls, the solver computes candidate sets for every empty cell. The *minimum remaining values* (MRV) heuristic selects the cell with the fewest candidates:

$$\text{MRV}(r, c) = \arg \min_{(r, c)} |\text{Candidates}(r, c)|.$$

This dramatically reduces branching and is essential for scaling to large N .

6 Shallow Branching with Forbidden-Value Learning

The solver performs only one level of branching:

1. Select the MRV cell.
2. Try each candidate value.
3. After each attempt, run full layer propagation.
4. If a contradiction occurs, mark the value as forbidden.

If all candidates fail, the puzzle is declared unsolvable. This shallow branching is far more efficient than full backtracking, yet powerful enough to resolve puzzles that propagation alone cannot solve.

7 Conflict Detection

A contradiction is detected if:

- a row contains duplicate digits,
- a column contains duplicate digits,
- a box contains duplicate digits,
- an empty cell has zero candidates.

These checks prune the search space aggressively and prevent wasted branching.

8 Full-Grid Generation

The generator constructs a valid Latin-based full solution using the formula:

$$\text{grid}[r][c] = (r \cdot b + \lfloor r/a \rfloor + c) \bmod N + 1,$$

where (a, b) is the box shape. The grid is then randomized by:

- permuting digits,
- shuffling rows within row bands,
- shuffling columns within column bands.

9 Puzzle Generation for Large N

Random clue removal fails for large grids such as 36×36 . The attached implementation uses:

- **symmetry-preserving removal** (180° rotational pairs),
- **minimum clues per row, column, and box**,
- **delayed solvability checks** (only near the target clue count),
- **the hybrid solver itself** to verify solvability.

This produces solvable puzzles efficiently even for 36×36 .

10 Performance Comparison

We compare three solver types:

10.1 Pure Backtracking

Backtracking explores a search tree of size approximately

$$O(N^{N^2}),$$

which is infeasible for $N > 9$.

10.2 Human-Style Logical Solvers

These rely on pattern recognition and do not generalize well to large N . Many rules become ineffective when candidate sets are large.

10.3 Hybrid Layer–MRV Method (This Work)

The attached solver reduces the search space by:

- eliminating most candidates through layer propagation,
- selecting the most constrained cell via MRV,
- performing only shallow branching,
- pruning contradictions early.

Empirical performance from the implementation:

- 9×9 : milliseconds,
- 12×12 : under a second,
- 16×16 : seconds,
- 36×36 : seconds to tens of seconds depending on clue density.

11 Conclusion

The solver and generator described here provide a practical and scalable approach to Sudoku of arbitrary size. The combination of layer propagation, MRV heuristics, shallow branching, and early conflict detection achieves a balance between speed and generality that is not present in classical approaches. The generator’s structure-aware clue removal ensures solvability even for large grids such as 36×36 .